Clasificación de hojas de árboles frutales con redes neuronales convolucionales

Javier Ruano Hernández - Hugo González Fernández

Universidad de Las Palmas de Gran Canaria (ULPGC)

Aprendizaje Automático II, curso 2026-2027

Repositorio en GitHub

23 de octubre de 2026

Resumen

En este trabajo abordamos la clasificación de hojas escaneadas de siete especies de árboles frutales a partir de un subconjunto de LeafScans-Orchard (2138 imágenes) y un desbalance de aproximadamente 6:1 entre clases. Hemos desarrollado en PyTorch una red neuronal convolucional (CNN) entrenada desde cero y la hemos mejorado a lo largo de nueve experimentos. En la fase manual, cinco configuraciones elevaron la exactitud en prueba del 84,1 % al 87,5 %, sobre todo al aumentar la resolución de entrada a 128×128 píxeles con un redimensionado que conserva la forma de las hojas. En la fase automática, una búsqueda con Optuna en dos etapas, de lo general a lo particular, condujo a una red de cuatro bloques convolucionales y 5,27 millones de parámetros que alcanza una exactitud media del 90,9 % $\pm$ 1,9 % sobre tres semillas. Por último, el ajuste fino de una EfficientNet-B0 preentrenada en ImageNet, con 4,0 millones de parámetros, eleva la exactitud media al 98,4 % $\pm$ 0,5 % y resuelve en gran medida la clase más difícil, el manzano.

Palabras clave: redes neuronales convolucionales, clasificación de hojas, optimización de hiperparámetros, Optuna, aprendizaje por transferencia, EfficientNet.

I. Introducción

La identificación automática de especies vegetales a partir de imágenes de sus hojas tiene aplicaciones directas en agronomía, como la gestión de explotaciones frutícolas, el inventario de variedades o la verificación del material vegetal en viveros [1]. Las hojas son accesibles durante gran parte del año y se digitalizan fácilmente con un escáner plano.

Sin embargo, el problema no es trivial. El ciruelo europeo, el melocotonero, el albaricoquero, el cerezo dulce y el guindo pertenecen al género *Prunus*, y el manzano (*Malus*) y el peral (*Pyrus*) están estrechamente emparentados. Sus hojas comparten forma general y color, y a menudo solo se distinguen por detalles finos como la proporción entre largo y ancho, la forma de la base o el dentado del margen.

Abordamos el problema con redes neuronales convolucionales (CNN) sobre un subconjunto de LeafScans-Orchard [2] de 2138 hojas de siete especies, organizando el trabajo en tres

fases: una **fase manual** (Exps. 1–5) que parte de la arquitectura de referencia de la asignatura y modifica pocos elementos cada vez; una **fase automática** (Exps. 6–8) en la que Optuna dimensiona nuestra CNN en dos etapas y la evaluamos con varias semillas; y un experimento de **aprendizaje por transferencia** (Exp. 9) con EfficientNet-B0 [3] preentrenada en ImageNet, como referencia frente a nuestro mejor modelo propio.

II. Conjunto de datos

II-A. Origen, características y partición

Los datos proceden de LeafScans-Orchard [2]: escaneos RGB de hojas individuales sobre fondo uniforme, tomados entre 2015 y 2025 y distribuidos en TIFF sin pérdidas a 300 ppp. Hemos tomado 400 imágenes por clase de manzano, ciruelo europeo, peral, guindo y cerezo dulce, y todas las disponibles de albaricoquero (74) y melocotonero (64). El subconjunto cumple las restricciones de la asignatura: datos reales, entre 5 y 15 categorías y al menos 50 imágenes por categoría. Tres características condicionan el trabajo: las imágenes no tienen tamaño fijo (lado mayor entre 343 y 2848 píxeles) y reflejan la forma alargada de las hojas; el manzano reúne 31 variedades frente a una o dos en el albaricoquero y el melocotonero, lo que lo convierte en la clase más difícil; y existe un desbalance de aproximadamente 6:1.

Dividimos los datos de forma estratificada (70 % entrenamiento, 15 % validación y 15 % prueba) con una semilla fija (42), lo que da 1497, 321 y 320 imágenes (Fig. 1). La partición es la misma en los nueve experimentos, de modo que todos los modelos se evalúan sobre las mismas 320 imágenes y pueden compararse imagen a imagen; el conjunto de prueba no se ha usado nunca para elegir hiperparámetros ni modelos. Como la exactitud global puede ocultar fallos en las clases pequeñas, informamos también del F1 macro. La prueba solo contiene 11 hojas de albaricoquero y 9 de melocotonero, por lo que sus métricas por clase deben leerse con cautela.

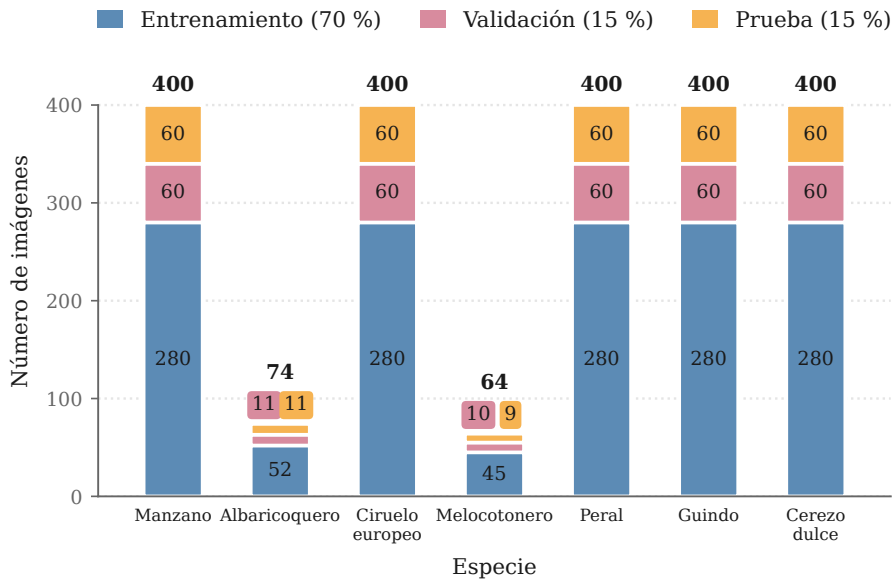


Figura 1: Distribución de imágenes por especie y partición. Se aprecia el desbalance de aproximadamente 6:1 entre las clases mayoritarias y el melocotonero.

II-B. Balanceo y sesgos de adquisición

Para compensar el desbalance aplicamos sobremuestreo en entrenamiento: cada clase se completa con copias aleatorias de sus propias imágenes hasta las 280 de la clase mayoritaria, lo que da $7 \times 280 = 1960$ muestras por época a partir de 1497 imágenes reales (en el melocotonero, cada hoja aparece unas seis veces por época). El aumento de datos (Sección III-B) evita que las copias sean idénticas. Validación y prueba no se balancean ni se aumentan.

Los escaneos de 2015 tienen fondo blanco y orientación horizontal, y los de 2022–2025, fondo gris oscuro y orientación casi siempre vertical. Como la proporción de imágenes de 2015 varía entre el 78 % del melocotonero y el 4 % del cerezo dulce, fondo y orientación podrían servir de atajo (*shortcut learning*). Las rotaciones del aumento de datos neutralizan la orientación, pero no el color del fondo (Sección VI-D).

III. Metodología

III-A. Preprocesamiento: redimensionado con relleno

En la primera versión del proyecto, las imágenes se redimensionaban directamente a 96×96 píxeles, lo que comprimía la mayoría de las hojas a la mitad a lo largo de su eje mayor y destruía su forma, una de las principales pistas para distinguir las especies (Fig. 2). Lo hemos sustituido por un redimensionado con relleno (*letterbox*): la imagen se escala para que su lado mayor mida S píxeles sin alterar la proporción y se centra en un lienzo de $S \times S$ rellenado con el color del fondo, estimado como la mediana de las cuatro esquinas. Como el fondo es uniforme, el relleno no crea bordes artificiales. Los Exps. 1–3 usan $S = 96$, como la arquitectura de referencia, y del Exp. 4 en adelante, $S = 128$. Después, los píxeles se normalizan con la media y la desviación típica de ImageNet, imprescindibles para EfficientNet-B0, de modo que todos los modelos reciben las mismas entradas.

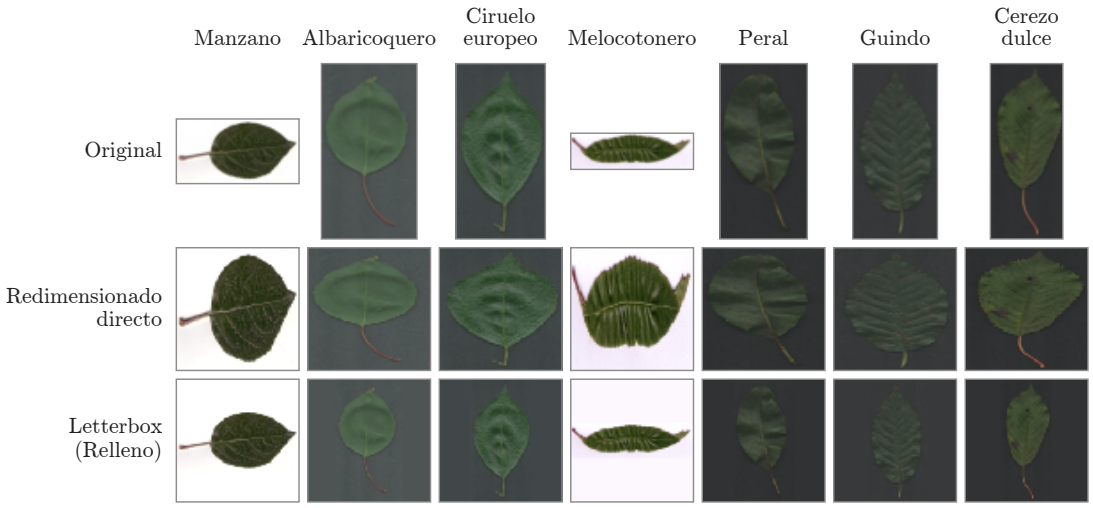


Figura 2: Efecto del preprocesado sobre la forma de las hojas. El redimensionado directo a un cuadrado deforma las hojas alargadas; el redimensionado con relleno conserva su proporción original.

III-B. Aumento de datos

Solo en entrenamiento y de forma aleatoria en cada carga aplicamos volteos horizontal y vertical, rotaciones de $k \cdot 90°$ y una variación leve del color (brillo, contraste y saturación $\times[0{,}8;\ 1{,}2]$ y tono $\pm 0{,}02$). En un escáner la hoja puede colocarse en cualquier orientación, así que estas transformaciones generan ejemplos realistas; las rotaciones de 90° son exactas y no crean esquinas de relleno, y el tono apenas se modifica porque el color de la hoja puede ser informativo.

III-C. Arquitectura propia

La arquitectura de referencia de la asignatura tiene tres bloques convolucionales (16, 32 y 64 filtros de 5×5, cada uno con ReLU y *max pooling* de 2×2), un aplanado de 9216 valores, una capa densa de 128 neuronas con *dropout* de 0,25 y una salida de 7 clases: 1 245 991 parámetros a 96×96. La hemos encapsulado en un modelo paramétrico que permite variar el número de bloques y filtros, el *kernel*, la capa densa, el *dropout*, la activación, la normalización por lotes y la cabeza de clasificación (aplanado o *Global Average Pooling*, GAP).

III-D. Entrenamiento y parada temprana

Todos los modelos, incluida EfficientNet-B0 y cada prueba de Optuna, se entrenan con el mismo bucle: entropía cruzada sin pesos por clase (el entrenamiento ya está balanceado), Adam sin regularización L2 y barajado con semilla. Para evitar el sobreajuste, al final de cada época se calcula la pérdida en validación; si mejora, se guardan los pesos, y si no mejora durante 7 épocas seguidas, el entrenamiento se detiene y se restauran los pesos de la mejor época. Usamos la pérdida y no la exactitud porque, con 321 imágenes, la exactitud varía en escalones de aproximadamente el 0,3 % y es mucho menos sensible.

III-E. Optimización de hiperparámetros

La fase manual mostró que los hiperparámetros interactúan y que el espacio de combinaciones es demasiado grande para explorarlo a mano. Por ello usamos Optuna, que guía la búsqueda con un estimador de Parzen estructurado en árbol (TPE), permite podar pruebas poco prometedoras y estima la importancia de cada hiperparámetro con fANOVA. La búsqueda se hizo en dos etapas (Tabla 1), minimizando en ambas la menor pérdida en validación de cada prueba. La primera (Exp. 6) explora a la vez arquitectura y optimización con resolución de 128 píxeles, cabeza de aplanado y GELU, con poda por mediana y un límite de 10 millones de parámetros. La segunda (Exp. 7) fija la mejor arquitectura y afina solo la tasa de aprendizaje y el *dropout*, con más épocas y sin poda para que las pruebas sean comparables. Cada estudio se guarda en SQLite, lo que permite interrumpir y reanudar la búsqueda.

Tabla 1: Espacio de búsqueda de las dos etapas de optimización con Optuna. Los rangos marcados con *escala log.* se muestrean en escala logarítmica.

Hiperparámetro	Etapa 1: búsqueda automática (Exp. 6)	Etapa 2: refinamiento (Exp. 7)
Bloques convolucionales	{3, 4, 5}	4 (fijo)
Filtros del primer bloque	{16, 32, 48}, duplicados en cada bloque (máx. 256)	32 (fijo): 32, 64, 128 y 256
Tamaño del kernel	{3, 5}	5 (fijo)
Normalización por lotes	{sí, no}	sí (fijo)
Neuronas de la capa densa	{128, 256, 512}	256 (fijo)
Dropout	[0,20; 0,60]	[0,15; 0,40]
Tasa de aprendizaje	$[10^{-4}; 2 \cdot 10^{-3}]$, escala log.	$[2 \cdot 10^{-5}; 3 \cdot 10^{-4}]$, escala log.
Tamaño de lote	{32, 64}	32 (fijo)
Épocas máximas	40	80
Pruebas aleatorias iniciales	10	4
Poda	Mediana (tras 5 pruebas completas, desde la época 5)	Sin poda
Límite de parámetros	10 millones	–
Pruebas previstas	40	8

III-F. Aprendizaje por transferencia

Para el Exp. 9 elegimos EfficientNet-B0 [3] entre las redes de `torchvision` por tres razones: su tamaño es comparable al de nuestra red (4,0 frente a 5,27 millones de parámetros, mientras que ResNet-18 tiene 11,2), de modo que la comparación mide el efecto del preentrenamiento y no del tamaño; con entradas de 128×128 conserva un mapa de 64×64 tras su primer bloque, mientras que ResNet-18 baja a 32×32 y puede borrar detalles como el dentado del margen; y tiene la mejor exactitud en ImageNet de las candidatas (77,7 %, frente al 72,2 % de MobileNetV2). Sustituimos su última capa lineal por una de 7 salidas, partimos de los pesos `IMAGENET1K_V1` y ajustamos toda la red con Adam y una tasa de 10^{-4}. El resto del protocolo es idéntico al de nuestra red; mantener 128 píxeles, en lugar de los 224 del preentrenamiento, garantiza entradas idénticas para ambos modelos, aunque puede no aprovechar del todo la red preentrenada.

IV. Configuración experimental

Los experimentos se ejecutaron en CPU (AMD Ryzen AI 5 330, sin CUDA), lo que supuso un alto coste computacional (unas 9 horas solo para Optuna) y limitó el número de pruebas y semillas. El código se organiza en `src/` y `experiments/`; cada ejecución guarda en `results/` su configuración y métricas, los pesos, las gráficas y las predicciones, y las instrucciones para reproducirlo están en el README del repositorio. La Tabla 2 resume los nueve experimentos.

Tabla 2: Resumen de los nueve experimentos: fase manual (Exps. 1–5), búsqueda con Optuna (Exps. 6 y 7) y modelos finales (Exps. 8 y 9). Todos usan Adam, entropía cruzada, parada temprana con paciencia de 7 y la misma partición de los datos. *Res.*: resolución de entrada en píxeles; BN: normalización por lotes.

Exp.	Modelo y cambio principal	Res.	Lote	Tasa de aprendizaje	Épocas	Semillas
1	Referencia: 16–32–64, ReLU, FC 128	96	32	10^{-3}	50	42
2	Más capacidad: filtros 32–64–128, FC 256	96	32	10^{-3}	50	42
3	Exp. 2 con activación GELU	96	32	10^{-3}	50	42
4	Exp. 3 a 128 px, lote mayor y tasa menor	128	64	$5 \cdot 10^{-4}$	50	42
5	Exp. 4 con GAP, BN y *ReduceLROnPlateau*	128	64	10^{-3}	50	42
6	Optuna, búsqueda automática (16 pruebas)	128	32/64	10^{-4}–$2 \cdot 10^{-3}$	40	42
7	Optuna, refinamiento de tasa y *dropout*	128	32	$2 \cdot 10^{-5}$–$3 \cdot 10^{-4}$	80	42
8	Red final de Optuna, tres semillas	128	32	$5{,}5 \cdot 10^{-5}$	80	42–44
9	EfficientNet-B0 preentrenada, tres semillas	128	32	10^{-4}	50	42–44

V. Resultados

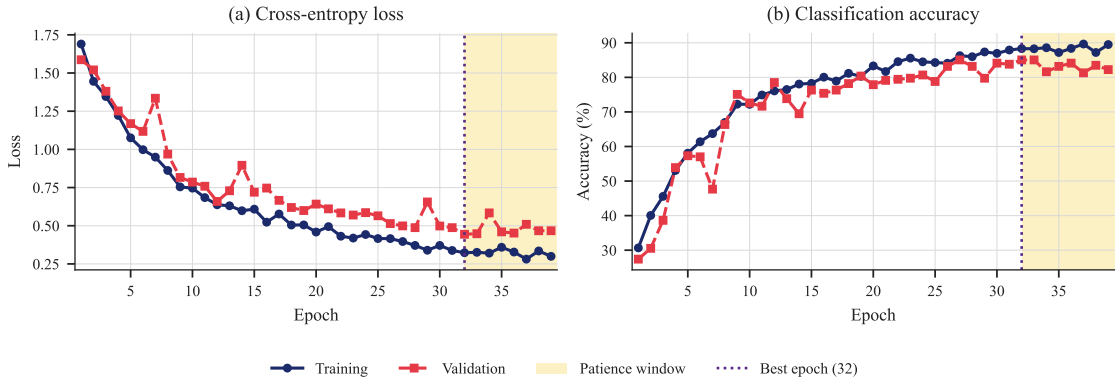
V-A. Del caso base al modelo final

La Tabla 3 resume las diferencias entre la red de referencia (Exp. 1), nuestra red final (Exp. 8) y EfficientNet-B0 (Exp. 9), y la Fig. 3 compara sus curvas de entrenamiento. Nuestro trabajo de diseño y ajuste lleva la exactitud en prueba del 84,1 % al 90,9 %, y la transferencia de conocimiento, hasta el 98,4 %. Las subsecciones siguientes explican cómo se llegó de un modelo a otro.

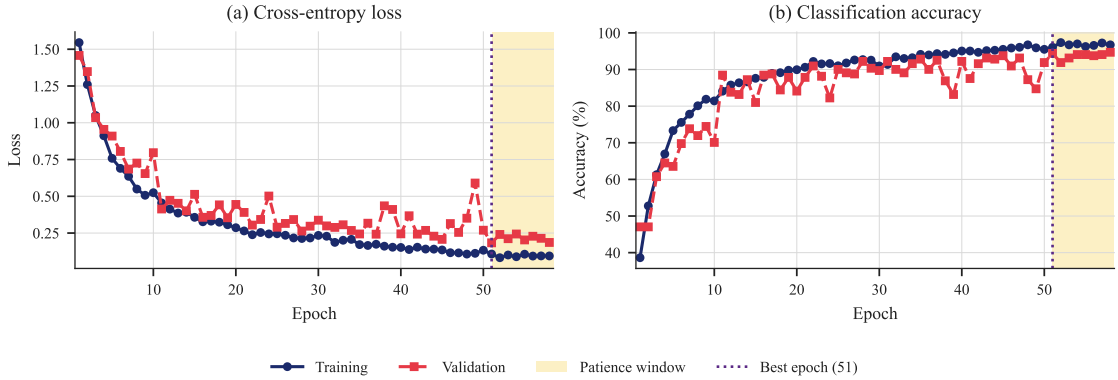
Tabla 3: Del caso base al modelo final. Exps. 8 y 9: media ± desviación típica de tres semillas y, entre paréntesis, la ejecución seleccionada.

	Referencia (Exp. 1)	CNN final (Exp. 8)	EfficientNet-B0 (Exp. 9)
Resolución	96×96	128×128	128×128
Bloques convolucionales	3 (16–32–64)	4 (32–64–128–256)	Preentrenados
Normalización por lotes	No	Sí	Sí
Activación	ReLU	GELU	SiLU
Capa densa y *dropout*	128; 0,25	256; 0,39	–; 0,2
Parámetros	1 245 991	5 274 919	4 016 515
Pesos iniciales	Aleatorios	Aleatorios	ImageNet
Tasa de aprendizaje	10^{-3}	$5{,}5 \cdot 10^{-5}$	10^{-4}
Épocas máximas	50	80	50
Exactitud en prueba	84,1 %	90,9 ± 1,9 % (92,2 %)	98,4 ± 0,5 % (98,75 %)
F1 macro en prueba	0,863	0,919 ± 0,023 (0,934)	0,986 ± 0,008 (0,990)
Sensibilidad del manzano	0,750	(0,833)	(0,950)
Errores / con manzano	51 / 37	(25 / 19)	(4 / 3)

(a) Exp. 1: red de referencia, 96 × 96, pesos aleatorios



(b) Exp. 8: red final, semilla 42, 128 × 128, pesos aleatorios



(c) Exp. 9: EfficientNet-B0, semilla 42, 128 × 128, pesos de ImageNet

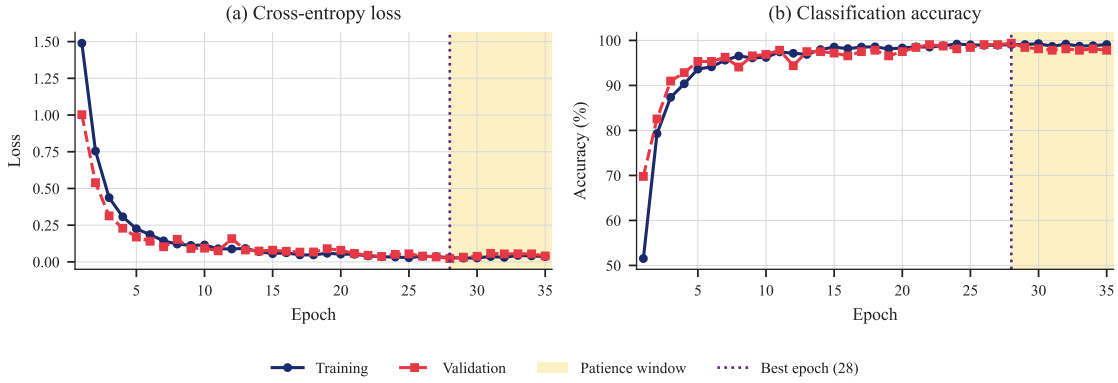
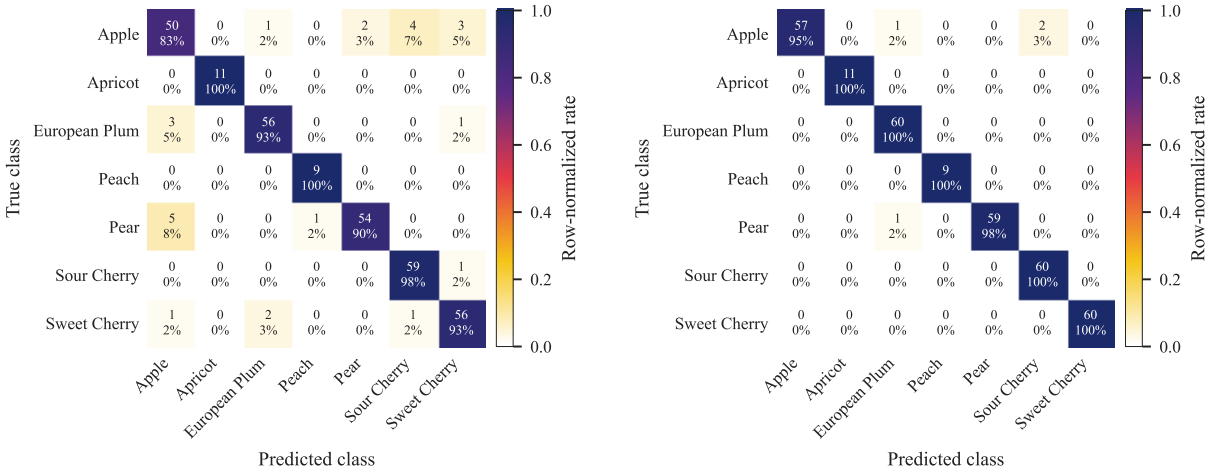


Figura 3: Curvas de pérdida y exactitud de entrenamiento (azul) y validación (rojo) de la red de referencia, la red final y EfficientNet-B0. La línea punteada marca la mejor época y la banda amarilla, la ventana de paciencia de la parada temprana. La red de referencia se estanca en torno al 85 % en validación; la red final sube de forma más lenta pero llega más alto, y EfficientNet-B0 supera el 95 % en validación en la época 5.

V-B. Análisis por clase

Para ver dónde se produce la mejora, comparamos las ejecuciones seleccionadas de los Exps. 8 y 9 mediante sus matrices de confusión (Fig. 4) y su F1 por clase junto al de la red de referencia (Tabla 4).



(a) CNN propia (Exp. 8): 295/320 aciertos.

(b) EfficientNet-B0 (Exp. 9): 316/320 aciertos.

Figura 4: Matrices de confusión normalizadas por filas sobre el conjunto de prueba para las ejecuciones seleccionadas de los Exps. 8 y 9. Los errores de nuestra red se concentran en torno al manzano; EfficientNet-B0 elimina casi todos.

Frente a la referencia, nuestra red mejora en todas las clases salvo el melocotonero, que se mantiene en 0,947 con solo 9 hojas en prueba, y sobre todo en el ciruelo europeo (F1 de 0,818 a 0,941) y el manzano (de 0,709 a 0,840). El manzano sigue siendo, aun así, la clase más difícil: 19 de los 25 errores del Exp. 8 lo implican, sobre todo perales clasificados como manzano (5) y manzanos clasificados como guindo (4) o cerezo dulce (3). EfficientNet-B0 elimina casi todos estos errores: de sus cuatro fallos, tres son manzanos y ningún peral se confunde ya con un manzano, y cinco de las siete clases alcanzan sensibilidad 1,0.

Tabla 4: F1 por clase en prueba de la red de referencia (Exp. 1) y de las ejecuciones seleccionadas de los Exps. 8 y 9.

Especie	Exp. 1	Exp. 8	Exp. 9
Manzano	0,709	0,840	0,974
Albaricoquero	0,917	1,000	1,000
Ciruelo europeo	0,818	0,941	0,984
Melocotonero	0,947	0,947	1,000
Peral	0,860	0,931	0,992
Guindo	0,887	0,952	0,984
Cerezo dulce	0,903	0,926	1,000
F1 macro	0,863	0,934	0,990
Sensibilidad del manzano	0,750	0,833	0,950
Errores totales / con manzano	51 / 37	25 / 19	4 / 3

VI. Discusión

VI-A. ¿Qué limitaba a la red de referencia?

El límite no fue el número de parámetros, sino la capacidad de *ver mejor la hoja*. Las mejoras clave fueron conservar su proporción (redimensionado con relleno), aumentar la resolución y usar una red más profunda con normalización por lotes para ampliar el campo receptivo. Se mantuvo la cabeza de aplanado porque el GAP promedia respuestas sobre un lienzo con mucho fondo, diluyendo detalles útiles (probable causa del fracaso del Exp. 5, junto a una baja paciencia del programador para reducir la tasa de aprendizaje).

VI-B. La varianza introducida por la parada temprana

La exactitud del Exp. 8 varía entre el 88,75 % y el 92,19 %. La causa principal es la interacción entre la parada temprana y una tasa de aprendizaje baja: la red mejora despacio, y la pérdida de validación, calculada sobre solo 321 imágenes, tiene picos frecuentes, por lo que encadenar 7 épocas sin un nuevo mínimo es en parte cuestión de suerte. Por eso un único entrenamiento no es un resultado y defendemos la media con su desviación típica.

VI-C. ¿Por qué EfficientNet-B0 resuelve el manzano?

El manzano es la clase más difícil por su gran variabilidad interna y su parecido con el peral. Entrenando desde cero con solo 280 imágenes, nuestra red se apoya en la forma global cambiante; en cambio, EfficientNet-B0 aprovecha sus filtros preentrenados para extraer detalles finos y estables (nervaduras, márgenes). La conclusión práctica es clara: con pocos datos, transferir conocimiento rinde mucho más que afinar una arquitectura desde cero.

VI-D. Amenazas a la validez

El color de fondo correlaciona con la clase y el año de escaneo, pudiendo actuar como atajo (ej. melocotonero). Además, la partición aleatoria mezcla sesiones y variedades entre entrenamiento y prueba, arrojando resultados quizá optimistas frente a hojas inéditas. Otras limitaciones son el tamaño reducido del conjunto de prueba, la reutilización de la validación para optimizar, y el uso de solo tres semillas que genera una desviación típica aproximada.

VII. Conclusiones y trabajo futuro

El proyecto deja cuatro lecciones prácticas: conservar forma y resolución rinde más que añadir parámetros; la búsqueda automática es más eficaz por etapas; la varianza exige evaluar con múltiples ejecuciones; y el aprendizaje por transferencia es la mejor opción con pocos datos.

Como trabajo futuro proponemos: (1) partición agrupada por sesión y segmentación del fondo para evitar atajos; (2) análisis con mapas Grad-CAM para entender en qué se fijan los modelos; (3) entrenar EfficientNet-B0 a 224×224 desde cero; y (4) implementar una parada temprana más robusta.

Referencias

[1] J. Wäldchen and P. Mäder, "Plant species identification using computer vision techniques: A systematic literature review," *Arch. Comput. Methods Eng.*, vol. 25, no. 2, pp. 507–543, 2018.

[2] P. Chwietczuk, S. Lipiński, and P. Chwietczuk, "LeafScans-Orchard: A multi-year open RGB scan dataset of orchard plant leaves for species and cultivar classification," *Data*, vol. 11, no. 7, Art. no. 153, 2026, doi: 10.3390/data11070153.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

A. Declaración sobre el uso de Inteligencia Artificial

En cumplimiento de la normativa de la asignatura sobre el uso de Inteligencia Artificial (IA), se detalla el empleo de asistentes basados en *Large Language Models* (LLMs), concretamente Gemini y Claude Opus 5.5, durante este proyecto.

Forma de utilización

La IA se ha empleado exclusivamente como apoyo en tres áreas:

1. **Depuración de código:** Identificación de errores en *scripts* de PyTorch (ej. gestión de dimensiones de tensores en la capa de aplanado).
2. **Visualizaciones y gráficas:** Creación de *scripts* en Python (Matplotlib) para procesar datos de Optuna y exportar figuras vectoriales en PDF.
3. **Revisión de estilo:** Refinamiento ortotipográfico y corrección de la redacción académica para asegurar claridad y rigor técnico.

Ejemplos de consultas (prompts) y respuestas

A modo ilustrativo, se muestran dos interacciones representativas:

Ejemplo 1: Maquetación en LaTeX

- **Consulta:** *.Actualiza esta tabla en LaTeX usando shortstack en las cabeceras para reducir el ancho."*
- **Respuesta:** La IA devolvió el código refactorizado con `\shortstack` y `booktabs`, manteniendo intactos los datos originales.

Ejemplo 2: Duda metodológica

- **Consulta:** *"Los modelos tienen resultados ajustados. ¿Cómo compruebo si las diferencias son estadísticamente significativas?"*
- **Respuesta:** Explicación teórica del test de McNemar y un *script* usando `statsmodels` para aplicarlo a las matrices de confusión.